

GUÍA DE TRABAJO – ARREGLOS BIDIMENSIONALES

1. INFORMACIÓN DE LA ASIGNATURA

Asignatura:	Lógica y Representación I		
Código:	2554208	Tipo de Curso:	Obligatorio
Plan de Estudios:	5	Semestre:	1
Créditos: 3	TPS: 6	TIS: 3	TPT: 64 TIT: 32

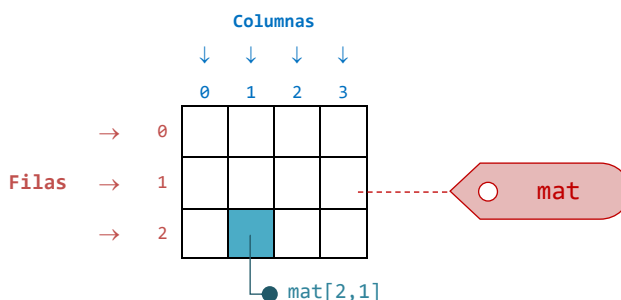
2. OBJETIVO

Unidad:	Estructuras de Datos Estáticas - Arreglos Bidimensionales
Objetivo de la asignatura:	Desarrollar en el estudiante la habilidad para plantear soluciones lógicas a problemas computacionales que deriven en la implementación de programas en un lenguaje de programación
Resultados de aprendizaje abordados:	- Usar las estructuras de datos para almacenar y manipular los datos asociados a un problema computacional - Escribir algoritmos y programas que solucionen problemas computacionales
Contenido temático:	- Introducción a los arreglos bidimensionales - Declaración, almacenamiento y acceso de datos en arreglos bidimensionales - Operaciones con arreglos bidimensionales - Ejemplos de aplicación de los arreglos de bidimensionales en programación

3. DESARROLLO TEÓRICO

3.1 DEFINICIÓN DE ARREGLOS BIDIMENSIONALES

En términos simples, un arreglo bidimensional (también conocido como matriz) se puede definir como una estructura rectangular que organiza un conjunto de datos en filas y columnas, formando una tabla. Cada elemento de la matriz está identificado por dos índices: uno que hace referencia a la fila y otro que señala la columna en la que está almacenado el dato; además, todos los elementos de la matriz comparten el mismo tipo de dato, bien sea primitivo (como entero, real o carácter) o un tipo de dato referencial (como Persona, Estudiante o Mascota). Gráficamente, una matriz se ve como sigue.

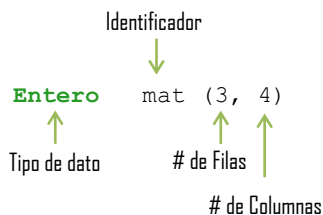


3.2 DECLARACIÓN DE ARREGLOS BIDIMENSIONALES

En la declaración de una matriz se debe especificar el `tipo_de_dato` y su tamaño, este en términos del número de filas y columnas. Igual que en los arreglos unidimensionales, el tamaño de un arreglo bidimensional se establece usando los paréntesis. En pseudocódigo, la forma general para declarar una matriz es como sigue:

```
<Tipo_de_dato> id_de_la_matriz (#_filas, #_columnas)
```

Por ejemplo, la matriz `mat`, que es de tamaño 3x4 y cuya representación gráfica está arriba, se declara así:



Importante: las filas de la matriz se distribuyen horizontalmente, mientras que las columnas se distribuyen verticalmente. Tenga presente que los índices de las filas y las columnas, al igual que los índices de un arreglo 1D, inician en el valor cero (0).

Análogo a como se hace en los arreglos unidimensionales, es posible declarar una matriz con valores predeterminados usando llaves, como se muestra en el siguiente ejemplo.

```
# Matriz de 2x3 con valores enteros predeterminados
Entero mat = {{1,2,3}, {4,5,6}}
```

En este ejemplo, se declara una matriz llamada `mat` de tamaño 2x3 en donde:

- La primera fila contiene los elementos {1, 2, 3}.
- La segunda fila contiene los elementos {4, 5, 6}.

Gráficamente, esta matriz se ve así:

	0	1	2
0	1	2	3
1	4	5	6

Ahora, declaremos una matriz, llamada `valores`, de tamaño 2x5, la cual almacena diferentes valores reales.

```
# Matriz de 2x5 con valores reales predeterminados
Real valores = {{2.5, 3.0, 4.8, 3.9, 4.2}, {4.2, 1.1, 3.7, 3.5, 3.6}}
```

	0	1	2	3	4
0	2.5	3.0	4.8	3.9	4.2
1	4.2	1.1	3.7	3.5	3.6

Como puede observarse, hay una relación entre los conjuntos de llaves usados en la declaración de la matriz y la representación gráfica, tal que cada llave interna contiene la lista de elementos de cada fila.

3.3 ACCESO A LOS ELEMENTOS DE UN ARREGLO BIDIMENSIONAL

Para acceder a los elementos de un arreglo bidimensional se usa el operador corchete [], indicando el índice de la fila y el índice de la columna que corresponden al elemento que se quiere acceder. Por ejemplo, para acceder al elemento que está en la fila *i* y columna *j* de la matriz *mat*, se utiliza la notación *mat[i,j]*. Veamos un ejemplo.

```
# Iniciamos declarando una matriz de 3x2 que tiene unos valores enteros predeterminados
Entero mat = {{1,2}, {3,4}, {4,5}}
```

Gráficamente, esta matriz se ve así:

		0	1
	0	1	2
mat =	1	3	4
	2	5	6

Vamos a realizar un par de modificaciones sobre los elementos de esta matriz, accediendo a algunos de sus elementos, así:

```
# Reemplacemos el 4 que está en la celda con fila 1 y columna 1, por un 9
mat[1, 1] = 9

# Para sumar 20 a la celda de la fila 2 y columna 0, lo hacemos así
mat[2, 0] = mat[2, 0] + 20

# Aquí sumamos dos celdas de la matriz, guardando el resultado en la fila 0 con columna 0
mat[0, 0] = mat[0, 1] + mat[1, 0]
```

Después de realizar estas operaciones la matriz queda así:

		0	1
	0	5	2
mat =	1	3	9
	2	25	6

Diagrama de anotaciones:

- *mat[0,0]* (verde) apunta a la celda (0,0) con valor 5.
- *mat[1,1]* (púrpura) apunta a la celda (1,1) con valor 9.
- *mat[2,0]* (azul) apunta a la celda (2,0) con valor 25.



Importante: Siempre tenga cuidado con el manejo de los índices pues si intenta acceder a una celda que no existe, se generará un error en tiempo de ejecución. También recuerde que es diferente tratar de acceder a la celda [2, 1] que a la celda [1, 2]. En el ejemplo anterior, la celda [2, 1] tiene el valor 6, mientras que la celda [1,2] NO existe.

3.4 EJEMPLOS CON ARREGLOS BIDIMENSIONALES

Las principales operaciones con matrices requieren recorrer las matrices, bien sea para almacenar información en estas o para acceder esa información. Veamos un ejemplo simple.



Ejemplo – Recorrido de matrices

- Desarrolle un algoritmo que almacene la información de dos matrices enteras, denominadas A y B, de tamaño 2x3. Después, cree una matriz denominada C, que contenga la suma, elemento a elemento, entre las matrices A y B.

Cuando necesitamos recorrer una matriz debemos utilizar dos ciclos, uno anidado en el otro. Uno de esos ciclos es para controlar el índice de las filas y el otro para controlar el índice de las columnas. Para resolver este problema, primero se pide al usuario la información de la matriz A, después pediremos la información de la matriz B, y finalmente sumaremos A y B, y almacenaremos el resultado en C.

```
# Declaramos las variables que requerimos
Entero A(2,3), B(2,3), C(2,3), i, j

# Usamos dos ciclos, uno anidado en el otro para recorrer A
# El primer ciclo controla las filas, por eso su límite es 2
# El segundo ciclo controla las columnas, por eso su límite es 3
Para i=0 hasta 2 Haga
  Para j=0 hasta 3 Haga
    Escribir("Ingrese el valor de A[" , i , " , " , j , "]: ")
    Leer(A[i,j])
  Fin_Para
Fin_Para

# Hacemos lo mismo con la matriz B
Para i=0 hasta 2 Haga
  Para j=0 hasta 3 Haga
    Escribir("Ingrese el valor de B[" , i , " , " , j , "]: ")
    Leer(B[i,j])
  Fin_Para
Fin_Para

# Ahora calculamos la suma de las matrices A y B, y almacenamos el resultado en C
Para i=0 hasta 2 Haga
  Para j=0 hasta 3 Haga
    C[i,j] = A[i,j] + B[i,j]
    Escribir("El valor de C[" , i , " , " , j , " es ", C[i,j])
  Fin_Para
Fin_Para
```

En este ejemplo, la expresión $C[i,j] = A[i,j] + B[i,j]$ almacena en la celda de la fila i y columna j de la matriz del C la suma de esa misma celda pero en las matrices A y B.



Importante: en este ejemplo el ciclo externo, que controla al índice i , permite recorrer todas las filas de la matriz, mientras que el ciclo interno, que controla el índice j , permite recorrer todas las columnas. Si observa como van cambiando los índices notará que esta forma de organizar los ciclos permite recorrer la matriz por filas, puesto que para cada fila i (fijada en el ciclo externo), se mueve el puntero por las columnas usando el índice j del ciclo interno.

Ahora considere la siguiente cuestión, ¿qué debemos hacer para mostrar todos los datos que hay sólo en la fila 0 de la matriz C?. En este caso, el número de la fila lo debemos fijar en 0 y utilizar un ciclo para movernos por las columnas de esa fila, así:

```
# Así mostramos el contenido de la fila 0 de la matriz C
Para j=0 hasta 3 Haga
  Escribir("El valor de C[" , 0 , " , " , j , " es ", C[0,j])
Fin_Para
```



Ejemplo – Recorriendo un Arreglo

- Desarrolle un algoritmo que muestre los números pares que se han almacenado en el arreglo `x` del ejemplo anterior.

Para solucionar este problema debemos recorrer el arreglo, es decir, debemos acceder a cada elemento del arreglo para verificar si el valor almacenado en él es par para mostrarlo. Como necesitamos acceder a todos los elementos del arreglo, debemos hacer uso de un ciclo.

```
# En este caso usaremos un ciclo Mientras, para variar :)
Escribir("Los número pares que hay en el arreglo son: ")
i=0
Mientras (i<10) Haga
    # Accedemos a la posición i del arreglo para verificar si el número es par y mostrarlo
    Si (x[i] MOD 2 == 0) Entonces
        Escribir(x[i])
    Fin_Si
    i=i+1
Fin_Mientras
```

En este caso, para mostrar un elemento del arreglo debemos cerciorarnos de que el elemento es par. Es por esto que usamos un condicional en el que verificamos si el residuo de la división del elemento `x[i]` con 2 es igual a 0.



Ejemplo – Encontrando el elemento de mayor valor

- Desarrolle un algoritmo que muestre cuál es el mayor valor almacenado en el arreglo e indique en qué posición está este

Este es un problema clásico que tiene muchas aplicaciones cuando se trabaja con arreglos. La idea principal es buscar el número más grande del arreglo y la posición que ocupa. Como debemos **buscar**, eso implica recorrer el arreglo y por tanto necesitamos un ciclo. Adicionalmente, se requieren dos variables: una para almacenar el mayor y otra para almacenar su posición. Veamos la solución.

```
# Primero declaramos las variables que requerimos
Entero valor_mayor, posicion_mayor, i

# En este caso usamos un ciclo mientras, por eso iniciamos la variable antes del ciclo
i=0

# Asumimos que el mayor está en la posición 0 del arreglo, y lo usaremos para las comparaciones
Valor_mayor = x[0]
Posición_mayor = 0

Mientras (i<10) Haga
    # Si el elemento i es mayor que valor_mayor
    # Lo reemplazamos porque hemos encontrado un nuevo mayor
    Si (x[i] > valorMayor) Entonces
        valor_mayor = x[i]
        posicion_mayor = i
    Fin_Si
    i=i+1
Fin_Mientras

# Al final mostramos el mayor y su posición:
Escribir("El mayor valor almacenado en el arreglo es ", valor_mayor)
Escribir("Y este se encuentra en la posición ", posicion_mayor , " del arreglo")
```

4. EJERCICIO PROPUESTO

XXXXXX

Analice el enunciado y con base en él:

- Defina el cliente, el usuario y las entidades del mundo del problema que intervienen en la solución
- Defina los requerimientos que deben ser implementados para construir una solución al problema
- Describa los atributos y métodos que deben tener las clases que representan las entidades del mundo del problema
- Con base en la descripción anterior, bosqueje el diagrama de clases de la solución
- Implemente la solución en pseudocódigo o algún lenguaje de programación orientado por objetos

5. BIBLIOGRAFÍA

- Herrera, A., Ebratt, R., & Capacho, J. (2016). Diseño y construcción de algoritmos. Barranquilla: Universidad del Norte.
- Aguilar, L. (2020). Fundamentos de programación: algoritmos, estructuras de datos y objetos (5a.ed.). México: Mc Graw Hill.
- Thomas Mailund. Introduction to Computational Thinking: Problem Solving, Algorithms, Data Structures, and More, Apress, 2021.
- Steven S. Skiena. The Algorithm Design Manual. Third Edition, Springer, 2020.
- Samuel Tomi Klein. Basic Concepts in Algorithms, World Scientific Publishing, 2021.

Elaborado Por:	Carlos Andrés Mera Banguero
Versión:	1.0
Fecha	Mayo de 2024